

CST8509 Assignment 2 Create 3 Red Ball

Individual Assignment

This is an individual assignment. You are not allowed to work together with anyone to produce any portion of your solution. You are not allowed let other students view your solution, and you are not allowed to view any portion of another student's solution. If your solution is influenced by a resource like ChatGPT or a web page, be sure to acknowledge that resource in your submission.

Stages

There are roughly four stages to this work:

1. Barebones Gymnasium environment working with null agent
2. Build on Stage 1 above to complete the workings of the Gymnasium environment
3. Implement the various agents to train on the environment
4. Fine tune the Environment based on results of Stage 3

Overview

We explore using RL to train a real-life physical robot using a simulation. We use the results of Lab 5 to continue to explore Reinforcement Learning for training the Create 3 robot to rotate to keep the moving red ball in the center of its field of view. The assignment is to create a Gymnasium environment based on the Lab 5 Gazebo simulation that can be trained with our own Q-learning code or Stable-Baselines3 algorithms.

This work has strong parallels to the BlocksWorld Assignment¹. The main difference is that instead of the Gymnasium environment interacting with the Prolog interpreter, the Gymnasium environment in this case will interact with a ROS 2 Node (similar to the ROS 2 Node from Lab 5) that can publish Twist messages and observe the results by subscribing to the virtual camera.

While completing this assignment, you will

- Create a Gymnasium environment **aisd_examples/CreateRedBall-v0**
- Implement Q-Learning on the CreateRedBall Gymnasium environment
- Apply a selection of Stable-Baselines3 algorithms to the Create3RedBall Gymnasium environment
- Implement a non-RL agent for comparison purposes that will compute the action without doing RL

Instructions

This work should be completed on your Ubuntu 22.04 loaner laptop with the GUI enabled (ubuntu-desktop package installed). Most students will find this work is NOT possible on their Ubuntu 22.04 virtual machine because of limited resources; however there is a chance it might work if it worked for Lab 5.

Like much of our work in this course, Assignment 2 involves a Gymnasium environment and various python RL agents that work on that environment. The simulation of Lab 5 with the red ball in the AWS small house must be running (and the Create 3 will be undocked) before you run your completed Assignment 2. Parts of the ROS 2 code from Lab 5 will be used in the new Gymnasium environment. We will not need to provide human-mode rendering with the new environment.

Create a folder for the assignment, Assn2, and put it under git control. The Assn2 folder will contain a new **aisd_examples** package (that contains Gymnasium environments only), as well as the various agent scripts. You will submit a zipped Assn2 folder. The final directory structure should look similar to this:

```
$ mkdir Assn2
$ cd Assn2
$ git init # this will create the local git repository
```

```
Assn2/
.git/      # git repository for committing while you work on the assignment
null.py    # null agent
non-rl.py  # agent that computes actions without using RL
qlearning.py # qlearning agent
ppo.py     # agent that uses PPO algorithm
dqn.py     # agent that uses DQN algorithm
<Algonquin_userid_not_student_number>_aisd_examples
pyproject.toml
aisd_examples/
  __init__.py
  envs/
    __init__.py
    create3_red_ball.py # the new environment for Assignment 2
    blocks_world.py    # the Assignment 1 environment not used in Assignment 2
```

Create 3 RedBall Gymnasium Environment

Create the package in the **Assn2/<Algonquin_userid_not_student_number>_aisd_examples** folder, and add/commit it to the git repository. Then using a text editor, create an initial

barebones version of the CreateRedBall-v0 Gymnasium environment's python file, **aisd_examples/envs/create3_red_ball.py** either by starting from scratch as in Lab 2, or alternatively by adapting a previous environment (BlocksWorld becomes CreateRedBall?). Adapting a BlocksWorld Gymnasium environment means changing the naming and stripping out the BlocksWorld code:

- `__init__` initializer: for now, set the `observation_space` and `action_space` to arbitrary Discrete spaces
- `reset`: for now, simply return any arbitrary state (int) from your observation space
- `step`: for now, simply return any arbitrary state (int) from your observation space, along with any arbitrary reward, terminate, truncate, info
- `render`: the render method can do nothing

Using a text editor, create the null agent, commit your work to your git repository, test the null agent to make sure it can call your **reset** and **step** methods without any problems, and fix any problems.

Commit your work to the git repository.

git commit -a -m "<insert meaningful message here> "

Now that the barebones Gymnasium environment has been lightly tested, it is time to add the python code to define the class for the RedBall ROS 2 Node into your Gymnasium environment python file by doing the following suggested steps

- Remember you are adding a class definition for a ROS 2 Node alongside the previously existing class definition which is for a Gymnasium environment class
- Start by copying the class definition (and not the **main** method) from the ROS 2 code you used in Lab5, and put that code in **envs/create_red_ball.py**
- The **main** method of the ROS 2 code you used in Lab 5 was (and is) necessary to instantiate the ROS Node at the beginning, and then tear down the ROS Node at the end. Your Gymnasium environment won't include the **main** method, but it will need to do what the main method does. The environment will keep a reference pointing to the ROS node, and we'll handle the code line by line. Here's the main method from the ROS 2 code:

```
def main(args=None):    # function definition not needed anymore

    rclpy.init(args=args)    # put this in the Gymnasium environment constructor, without args

    redball = RedBall()    # this initializes and keeps a reference to a ROS 2 node
                           # and it belongs in the Gymnasium
                           # environment initializer, where it is: self.redball = RedBall()

    rclpy.spin(redball)    # rclpy.spin does not return until shut down at the end, so instead
                           # we need to use rclpy.spin_once repeatedly to process ROS 2
                           # subscriptions. For now, we should put rclpy.spin_once(self.redball) in
                           # the Gymnasium environment step() method. Then when the step() method is
                           # repeatedly called by an agent, the RedBall node will spin.
```

```
# Destroy the node explicitly
# (optional - otherwise it will be done automatically
# when the garbage collector destroys the node object)
redball.destroy_node()      # this goes in the Gymnasium Environment close() method
rclpy.shutdown()            # this goes in the Gymnasium Environment close() method
```

After you have put the above code where it needs to be, commit your work to git.

git commit -a -m "<insert meaningful message here> "

Run the null agent (see Trial Run below). At this point, running the null agent will call **step()** on the environment repeatedly, and this causes **rclpy.spin_once()** to be called repeatedly. In Gazebo and Rviz you should see similar behavior to what you saw from Lab 5, because the ROS 2 code from Lab 5 is now running as part of the Gymnasium environment. In Rviz you should be able to display the **Image** from the **/target_redball** topic being published by the ROS 2 code that is now part of the Gymnasium environment.

Commit your work to the local .git repository with the following command (or equivalent git command):

git commit -a -m "target_redball topic being published by the ROS 2 code "

Trial run with barebones `aisd_examples/CreateRedBall-v0`

Use the following code as a “null agent” to test your barebones `CreateRedBall-v0`

```
import gymnasium as gym
import aisd_examples
env = gym.make("aisd_examples/CreateRedBall-v0", render_mode="human")
observation, info = env.reset()

for _ in range(1000):
    action = env.action_space.sample() # agent policy that uses the observation and info
    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

Filling out barebones `aisd_examples/CreateRedBall-v0`

At this point you have a minimal first draft of a Gymnasium environment based on the simulated Create 3, and we will now address its main limitations. Partial credit will be given if you partially address these limitations. We have the following points to cover:

- **States or observations:** think about what actually constitutes an observation in our environment. We cannot move the camera up and down, we can only turn left or right. Here are some candidates we can consider:
 - The entire image from the simulated camera
 - A floating point (real) number indicating where the red ball is in the image, from 0 on the far left, to 1 on the far right
 - An integer pixel value of where the red ball is in the image, ranging from 0 on the far left to 640 on the far right (our simulated camera image is 640x480 pixels)
 - In each of the above three cases, to ensure the Markov property we can consider multiples of the most recent 1, 2, 3, or 4, etc, of the mentioned item. For example, the most recent 2 images from the camera taken together could be considered the current state.

We will begin with a baseline implementation by considering the integer x-axis pixel value as the discrete observation.

- **Actions:** think about how we should represent actions, which will translate into Twist messages. Continuous actions (floating point numbers) are possible, but continuous actions cannot be used with qlearning, for example. Here are some candidates for how we can represent actions:
 - A floating point number between $-\pi/2$ and $\pi/2$, where $-\pi/2$ (minus 90 degrees) would translate into a Twist message that rotates 90 degrees to the left
 - -1, 0, +1, where -1 would translate to a Twist message that rotates “slightly” to the left
 - An integer between 0 and 640, where 320 (the middle of the camera image) would mean no rotation, and 0 would mean the largest rotation to the left

We will try the last option, where the action space is discrete. We can then translate actions into Twist messages by setting the angle component to something like $(A - 320)/320 * \pi/2$

This way, an action of 0 would mean an angular Twist component of $-\pi/2$, to turn to the left by 90 degrees. We should keep in mind that 90 degrees might be an excessive amount to turn, compared to the angular view of the camera.

- **Episodes and resetting:** Our robot has the never-ending task of turning to face the red ball as it moves. We will arbitrarily consider an episode to be 100 steps, so that each step increments a counter, and the episode terminates when the counter reaches 100. The reset method would set the counter to 0. This will work with our qlearning algorithm as well as the more advanced StableBaselines3 algorithms.
- **reset method:** given our definition of an episode, the reset method simply sets the counter to 0 and returns the most recent observation
- **step method:** The step method receives an integer action, causes a Twist message based on that action (see above), using the ROS 2 node to publish the Twist message.

After arranging for the publishing of the Twist message, the step method then repeatedly calls `rclpy.spin_once()` on the ROS 2 Node until the Create 3 wheels stop moving. As soon as the wheels stop moving, the step method returns an observation, which is the x-axis pixel value of the center of the red ball.

To implement this, you can add methods as necessary to your ROS 2 Node class.

Note that the Create 3 publishes `irobot_create_msgs.msg.StopStatus` messages on the `/stop_status` topic, and you can add a subscriber to that topic to your ROS 2 Node class, as well as its callback function. Your ROS 2 Node already subscribes to the virtual camera image and already has a callback to analyze the image.

Here is some sample Gymnasium environment **step** method code to give you ideas for methods to add to your ROS 2 Node (but you should not consider this to be the one and only correct way – corrections and improvements may be possible):

```
def step(self, action):
    self.redball.step(action)
    rclpy.spin_once(self.redball)
    while not self.redball.create3_is_stopped:
        rclpy.spin_once(self.redball)
    return self.redball.redball_position,
           self.reward(self.redball.redball_position),
           self.step_count == 100,
           False,
           {"info": None}
```

- **render method:** we won't support human rendering and this method can simply return
- **close method:** shut down the ROS 2 node the way the old main method would

After addressing the above limitations, commit your work to the local git repository.

[Q-Learning with aisd_examples/CreateRedBall-v0](#)

In `qlearning.py`, implement a Q-learning agent to work with your new CreateRedBall-v0 environment.

As with previous activities, keep track of returns per episode, and plot graphs at the end. Use “Original Hyperparameters” as the title of the plot. Take a screenshot, give it a meaningful name, and add it to your local git repository.

[Use your environment with Stable-Baselines3](#)

Try your environment with the **DQN** and **PPO** algorithms.

[Try your environment without RL](#)

Copy/enhance your null agent to do its own action choices without RL. Keep track of returns per episode, and plot a graph at the end. Take a screenshot, give it a meaningful name, and add it to your local git repository.

Be prepared during the demonstration to discuss the comparison between your qlearning graph and your non-RL graph.

Submission

Commit to your local git repository often, because you will be graded in part by how properly you used git source code control. Submit your zipped Assn2 folder which should have the structure shown above. The folder hierarchy should not include a python virtual environment folder, but it should include a local git repository folder (.git).

Demonstration

Are your states Markov States? If not, how could they be made to be Markov?

Show your Gymnasium Environment CreateRedBall-v0 working with your lab instructor's choice of Qlearning, PPO, or DQN

Show graphs of returns and steps per episode with your Qlearning algorithm and non-RL algorithm. Discuss the comparison between your qlearning graph and your non-RL graph.

Be prepared to answer questions about how your code works.